Verilog Language and Application

Week 6 Lab Document

Lab Manual **Revision 1.0**

cadence®

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Table of Contents

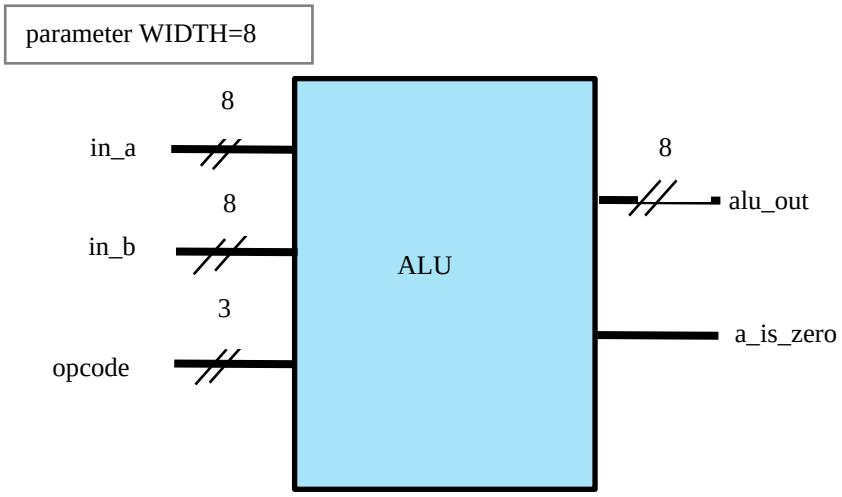
Verilog Language and Application

Lab 1 Modeling the Arithmetic Logic Unit

Objective: To use Verilog operators to describe a parameterized-width arithmetic logic unit (ALU).

ALU performs arithmetic operations on numbers depending upon the operation encoded in the instruction. This ALU will perform 8 operations on the 8-bit inputs (see table in the S*pecification*) and generate an 8-bit output and single-bit output.

In this lab, you create an ALU design as per the specification and verify it using the provided testbench. Please make sure to use the same variable name as the ones shown in block diagrams.



Specifications

- `in_a, in_b,` and `alu_out` are all 8-bit long. The `opcode` is a 3-bit value for the CPU operation code, as defined in the following table.
- `a_is_zero` is a single bit asynchronous output with a value of 1 when `in_a` equals 0. Otherwise, `a_is_zero` is 0.
- The output `alu_out` value will depend on the `opcode` value as per the following table.
- To select which of the 8 operations to perform, you will use `opcode` as the selection lines.
- The following table states the opcode/instruction, opcode encoding, operation, and output.

Opcode/ Instruction	Opcode Encoding	Operation	Output
HLT	000	PASS A	in_a => alu_out
SKZ	001	PASS A	in_a => alu_out
ADD	010	ADD	in_a + in_b => alu_out
AND	011	AND	in_a & in_b => alu_out
XOR	100	XOR	in_a ^ in_b => alu_out
LDA	101	PASS B	in_b => alu_out
STO	110	PASS A	in_a => alu_out
JMP	111	PASS A	in_a => alu_out

Designing an ALU

1. Change to the *lab1-alu* directory and examine the following files.

alu_test.v	ALU test
filelist.txt	File listing all modules to simulate

2. Use your favorite editor to create the *alu.v* file. Describe the ALU module named as *alu*.

   a. Parameterize the ALU input and output width so that the instantiating module can specify the width of each instance.

   b. Assign a default value to the parameter.

Verifying the ALU Design

1. Using the provided testbench module, check your ALU design using the following command with Xcelium™.

   ```
   xrun alu.v alu_test.v (Batch Mode)
   ```

   or

   ```
   xrun alu.v alu_test.v -gui -access +rwc ( GUI Mode)
   ```

2. You might find it easier to list all the files and simulation options in a text file and pass the file into the simulator using the `-f xrun` option.

```
xrun -f filelist.txt -access rwc
```

You should see the following results.

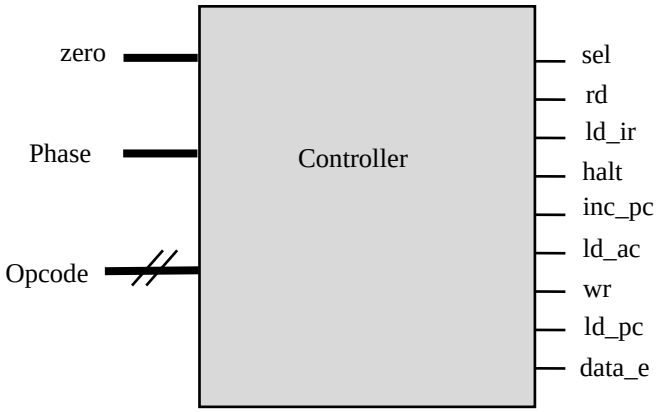
```
At time 1 opcode=000 in_a=01000010 in_b=10000110 a_is_zero=0
alu_out=01000010
At time 2 opcode=001 in_a=01000010 in_b=10000110 a_is_zero=0
alu_out=01000010
At time 3 opcode=010 in_a=01000010 in_b=10000110 a_is_zero=0
alu_out=11001000
At time 4 opcode=011 in_a=01000010 in_b=10000110 a_is_zero=0
alu_out=00000010
At time 5 opcode=100 in_a=01000010 in_b=10000110 a_is_zero=0
alu_out=11000100
At time 6 opcode=101 in_a=01000010 in_b=10000110 a_is_zero=0
alu_out=10000110
At time 7 opcode=110 in_a=01000010 in_b=10000110 a_is_zero=0
alu_out=01000010
At time 8 opcode=111 in_a=01000010 in_b=10000110 a_is_zero=0
alu_out=01000010
At time 9 opcode=111 in_a=00000000 in_b=10000110 a_is_zero=1
alu_out=00000000
TEST PASSED
```

3. Correct your ALU design as needed.

Lab 2 Modeling a Controller

Objective: To use the Verilog case statement to describe a controller.

The controller generates all control signals for the VeriRISC CPU. The operation code, fetch-and-execute phase, and whether the accumulator is zero determine the control signal levels.



Specifications

- The controller is clocked on the rising edge of `clk`.
- `rst` is synchronous and active high.
- `zero` is an **input** that is 1 when the CPU accumulator is zero and 0 otherwise.
- `opcode` is a 3-bit **input** for CPU operation, as shown in the following table.

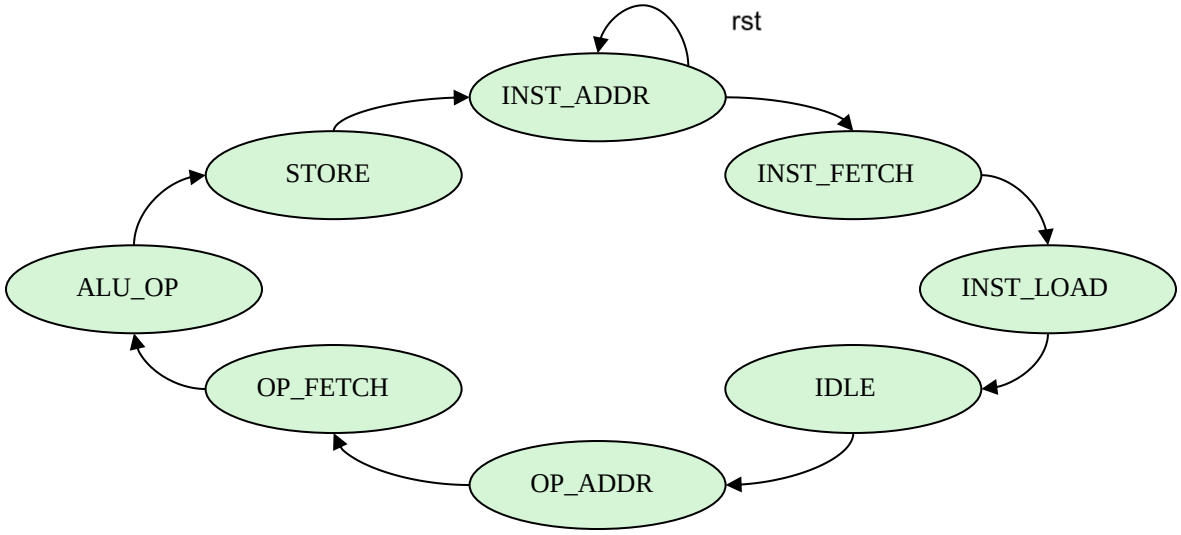
Opcode/ Instruction	Opcode Encoding	Operation	Output
HLT	000	PASS A	in_a => alu_out
SKZ	001	PASS A	in_a => alu_out
ADD	010	ADD	in_a + in_b => alu_out
AND	011	AND	in_a & in_b => alu_out
XOR	100	XOR	in_a ^ in_b => alu_out
LDA	101	PASS B	in_b => alu_out
STO	110	PASS A	in_a => alu_out
JMP	111	PASS A	in_a => alu_out

- There are 7 single-bit **outputs**, as shown in this table.

Output	Function
`sel`	select
`rd`	memory read
`ld_ir`	load instruction register
`halt`	halt
`inc_pc`	increment program counter
`ld_ac`	load accumulator
`ld_pc`	load program counter
`wr`	memory write
`data_e`	data enable

- The controller has a 3-bit phase input with a total of 8 phases processed. Phase transitions are unconditional which means `INST_ADDR, INST_ADDR.....STORE` any phase can come at any time according to the phase; all output signal values will be set according to the phase, which is shown in the table.

- The controller passes through the same 8-phase sequence, from `INST_ADDR` to `STORE`, every 8 `clk` cycles. The reset state is `INST_ADDR`.



- The controller outputs will be decoded w.r.t phase and opcode, as shown in this table.

Outputs	**Phase**						**Notes**		
	INST_ ADDR	INST_ FETCH	INST_ LOAD	IDLE	OP_ ADDR	OP_ FETCH	ALU_ OP	STORE	
`sel`	1	1	1	1	0	0	0	0	`ALU_OP = 1` if opcode is `ADD, AND, XOR` or `LDA`
`rd`	0	1	1	1	0	ALUOP	ALUOP	ALUOP	
`ld_ir`	0	0	1	1	0	0	0	0	
`halt`	0	0	0	0	HALT	0	0	0	
`inc_pc`	0	0	0	0	1	0	SKZ && zero	0	
`ld_ac`	0	0	0	0	0	0	0	ALUOP	
`ld_pc`	0	0	0	0	0	0	JMP	JMP	
`wr`	0	0	0	0	0	0	0	STO	
`data_e`	0	0	0	0	0	0	STO	STO	

Designing a Controller

1. Change to the *lab2-ctlr* directory and examine the following files.

controller_test.v	Controller test
filelist.txt	File listing all modules to simulate

2. Use your favorite editor to create the *controller.v* file and describe the controller module named "*control* ".

3. Declare a reg for each of these intermediate terms and establish their value immediately before entering the case statement.

Verifying the Controller Design

1. Using the provided test module, check your controller design using the following command with Xcelium™.

```
xrun controller.v controller_test.v (Batch Mode)
```

or

```
xrun controller.v controller_test.v -gui -access +rwc ( GUI Mode)
```

2. You might find it easier to list all the files and simulation options in a text file and pass the file into the simulator using the `-f xrun` option.

```
xrun -f filelist.txt -access rwc
```

You should see the following results.

```
Testing opcode HLT phase 0 1 2 3 4 5 6 7
Testing opcode SKZ phase 0 1 2 3 4 5 6 7
Testing opcode ADD phase 0 1 2 3 4 5 6 7
Testing opcode AND phase 0 1 2 3 4 5 6 7
Testing opcode XOR phase 0 1 2 3 4 5 6 7
Testing opcode LDA phase 0 1 2 3 4 5 6 7
Testing opcode STO phase 0 1 2 3 4 5 6 7
Testing opcode JMP phase 0 1 2 3 4 5 6 7
TEST PASSED
```

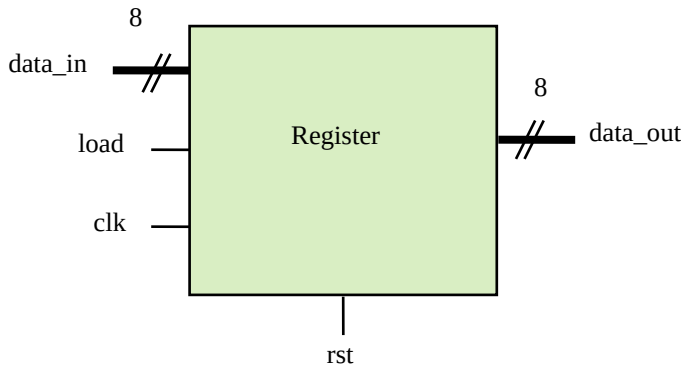
3. Correct your controller design as needed.

Lab 3 Modeling a Generic Register

Objective: To use nonblocking assignments to describe a register.

The VeriRISC CPU contains an accumulator register and an instruction register. One generic register definition can serve both purposes.

In this lab, you create a simple register design as per the specification and verify it using the provided testbench. Please make sure to use the same variable name as the ones shown in block diagrams.



Specifications

- `data_in` and `data_out` are both 8-bit signals.
- `rst` is synchronous and active high.
- The register is clocked on the rising edge of `clk`.
- If `load` is high, the input data is passed to the output `data_out`.
- Otherwise, the current value of `data_out` is retained in the register.

Designing a Register

1. Change to the *lab3-reg* directory and examine the following files.

register_test.v	Register test
filelist.txt	File listing all modules to simulate

2. Use your favorite editor to create the *register.v* file and to describe the register module named "*register* ".

   a. Parameterize the register data input and output width so that the instantiating module can specify the width of each instance.

   b. Assign a default value to the parameter as 8.

   c. Write the register model using the Verilog `always` procedural block.

Verifying the Register Design

1. Using the provided test module, check your register design using the following command with Xcelium™.

   ```
   xrun register.v register_test.v (Batch Mode)
   ```

   or

   ```
   xrun register.v register_test.v -gui -access +rwc ( GUI Mode)
   ```

2. You might find it easier to list all the files and simulation options in a text file and pass the file into the simulator using the `-f xrun` option.

   ```
   xrun -f filelist.txt -access rwc
   ```

   You should see the following results.

   ```
   At time 20 rst=0 load=1 data_in=01010101 data_out=01010101
   At time 30 rst=0 load=1 data_in=10101010 data_out=10101010
   At time 40 rst=0 load=1 data_in=11111111 data_out=11111111
   At time 50 rst=1 load=1 data_in=11111111 data_out=00000000
   TEST PASSED
   ```

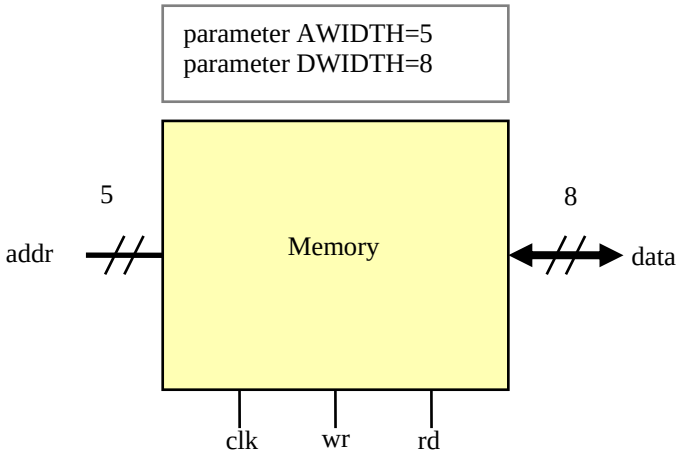
3. Correct your register design as needed.

Lab 4 Modeling a Single-Bidirectional-Port Memory

Objective: To use continuous and procedural assignments to describe a memory.

The VeriRISC CPU uses the same memory for instructions and data. The memory has a single bidirectional data port and separate write and read control inputs. It cannot perform simultaneous write and read operations.

In this lab, you create a simple register design as per the specification and verify it using the provided testbench. Please make sure to use the same variable name as the ones shown in block diagrams.

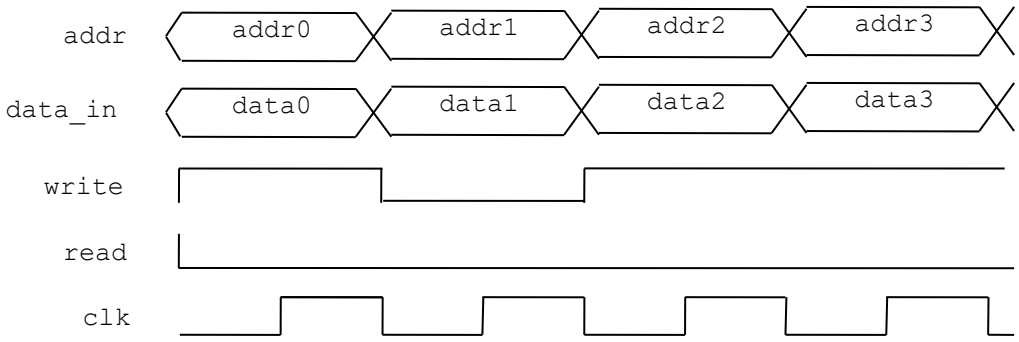


Specifications

- `addr` is parameterized to 5 and `data` is parameterized to 8.
- `wr` (write) and `rd` (read) are single-bit input signals.
- The memory is clocked on the rising edge of `clk`.
- Analyze the memory read and write operation timing diagram, as shown in the following graphic.

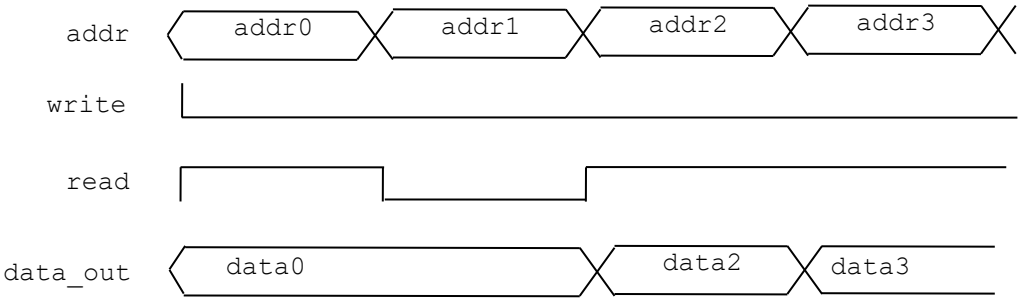
- **Memory write:** `data_in` is written to memory[addr] on the positive edge of `clk` when `wr` (write) =1.

Memory Write Cycle



- **Memory read**: `data_out` is assigned from memory[addr] when `rd` (read) =1.

Memory Read Cycle



Designing a Memory

1. Change to the *lab4-mem* directory and examine the following files.

memory_test.v	Memory test

2. Use your favorite editor to create the *memory.v* file and to describe the memory module named "*memory* ".

3. Parameterize the address and data widths so that the instantiating module can specify the width and depth of each instance.

4. Assign default values to the parameters.

5. Write data on the active clock edge when the *wr* input is true and drive data when the *rd* input is true.

6. Perform the write operation in a procedural block and perform the read operation as a continuous assignment.

Verifying the Memory Design

1. Using the provided test module, check your memory design using the following command with Xcelium™.

   ```
   xrun memory.v memory_test.v (Batch Mode)
   ```

   or

   ```
   xrun memory.v memory_test.v -gui -access +rwc ( GUI Mode)
   ```

2. You might find it easier to list all the files and simulation options in a text file and pass the file into the simulator using the `-f xrun` option.

   ```
   xrun -f filelist.txt -access rwc
   ```

   You should see the following results.

   ```
   At time 370 addr=11111 data=00000000
   At time 380 addr=11110 data=00000001
   At time 390 addr=11101 data=00000010
   At time 400 addr=11100 data=00000011
   At time 410 addr=11011 data=00000100
   At time 420 addr=11010 data=00000101
   At time 430 addr=11001 data=00000110
   At time 440 addr=11000 data=00000111
   At time 450 addr=10111 data=00001000
   At time 460 addr=10110 data=00001001
   At time 470 addr=10101 data=00001010
   At time 480 addr=10100 data=00001011
   At time 490 addr=10011 data=00001100
   At time 500 addr=10010 data=00001101
   At time 510 addr=10001 data=00001110
   At time 520 addr=10000 data=00001111
   At time 530 addr=01111 data=00010000
   At time 540 addr=01110 data=00010001
   At time 550 addr=01101 data=00010010
   At time 560 addr=01100 data=00010011
   At time 570 addr=01011 data=00010100
   ```



```
At time 580 addr=01010 data=00010101
At time 590 addr=01001 data=00010110
At time 600 addr=01000 data=00010111
At time 610 addr=00111 data=00011000
At time 620 addr=00110 data=00011001
At time 630 addr=00101 data=00011010
At time 640 addr=00100 data=00011011
At time 650 addr=00011 data=00011100
At time 660 addr=00010 data=00011101
At time 670 addr=00001 data=00011110
TEST PASSED
```

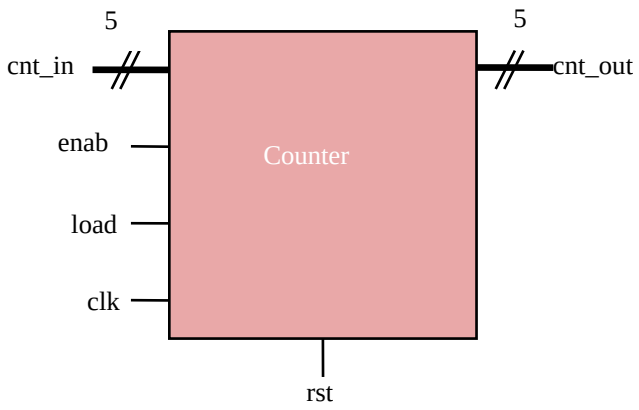
3. Correct your memory design as needed.

Lab 5 Modeling a Generic Counter

Objective: To use blocking and nonblocking assignments to describe a counter.

The VeriRISC CPU contains a program counter and a phase counter. One generic counter definition can serve both purposes.

In this lab, you create a simple register design as per the specification and verify it using the provided testbench. Please make sure to use the same variable name as the ones shown in block diagrams.



Specifications

- The counter is clocked on the rising edge of `clk`.
- `rst` is active high.
- `cnt_in` and `cnt_out` are both 5-bit signals.
- If `rst` is high, output will become *zero*.
- If `load` is high, the counter is loaded from the input `cnt_in`.
- Otherwise, if `enab` is high, `cnt_out` is incremented, and `cnt_out` is unchanged.

Designing a Generic Counter

1. Change to the *lab5-cntr* directory and examine the following files.

counter_test.v	Counter test

2. Use your favorite editor to create the *counter.v* file and to describe the counter module named as "*counter* ".

3. Parameterize the counter data input and output width so that the instantiating module can specify the width of each instance. Assign a default value to the parameter.

4. Describe the counter behavior in separate combinational and sequential procedures.

Verifying the Counter Design

1. Using the provided test module, check your counter design using the following command with Xcelium™.

```
xrun counter.v counter_test.v (Batch Mode)
```

or

```
xrun counter.v counter_test.v -gui -access +rwc ( GUI Mode)
```

2. You might find it easier to list all the files and simulation options in a text file and pass the file into the simulator using the `-f xrun` option.

```
xrun -f filelist.txt -access rwc
```

You should see the following results.

```
At time 20 rst=0 load=1 enab=1 cnt_in=10101 cnt_out=10101
At time 30 rst=0 load=1 enab=1 cnt_in=01010 cnt_out=01010
At time 40 rst=0 load=1 enab=1 cnt_in=11111 cnt_out=11111
At time 50 rst=1 load=1 enab=1 cnt_in=11111 cnt_out=00000
At time 60 rst=0 load=1 enab=1 cnt_in=11111 cnt_out=11111
At time 70 rst=0 load=0 enab=1 cnt_in=11111 cnt_out=00000
TEST PASSED
```

3. Correct your counter description as needed.